

Where Should the LLM Live? Three Placements for Language Models in Analytical Data Systems

Kamlesh Kumar
University of Helsinki
kamlesh.kumar@helsinki.fi

Abstract

I have read three papers [1–3] on the integration of Large Language Models with analytical database systems for Data Warehousing and Business Intelligence course and wrote a report. In this report, I review their main contributions on the problem of deploying LLMs in data warehousing and OLAP environments, and compare three distinct architectural approaches: keeping the LLM outside the analytical engine by retrieving precomputed OLAP cuboids [1], embedding aggressively compressed per-query LLMs directly inside the execution pipeline [2], and using the LLM solely to explain query performance differences in an HTAP system [3]. Beyond reviewing the three papers, I analyse weaknesses the authors leave unaddressed, in particular that none of the three systems has a mechanism for detecting or correcting errors at runtime and propose a feedback-loop architecture that wires the explanation channel of [3] into the calibration step of [2], a connection none of the three papers makes.

Keywords

LLM, OLAP, HTAP, Retrieval-Augmented Generation, instance optimisation, data warehousing

1 Introduction

The question I kept coming back to while reading these papers was narrower than the course topic suggests. “LLMs for data warehousing” could mean text-to-SQL, schema design, ETL automation, or a dozen other things. The three papers I chose share a more specific concern. Assume a real analytical workload is running on a real warehouse where in the stack should the language model live, and what should it be doing there? Put that way, the papers disagree, and the disagreement is substantive.

Ouafiq and Saadane [1] come at it from outside. LLMs are unreliable at generating correct SQL for non-trivial multi-dimensional aggregations, and RAG as commonly deployed was not designed for the kind of roll-ups OLAP requires. So they precompute the cuboids, store them in Cassandra with a bit-vector row-key encoding, and let a RAG pipeline fetch aggregated results rather than raw rows. The LLM orchestrates reasoning and produces explanations; it does not do the aggregation. On a 1M-record workload they report roughly 44% lower CPU consumption and around 62.5% less memory usage than a Hive star-schema baseline [1].

Mohammadi and Bindschaedler [2] go the opposite direction. They want the LLM inside the query itself, callable per row from SQL. The obvious objection is cost: a general-purpose 8B-parameter model invoked a billion times is not an option. Their answer, IOLM-DB, compiles a specialised compressed LLM for each query instance using quantisation, sparsification, and structural pruning, calibrated on a small sample of the query’s own data. On three workloads (summarising Amazon reviews, correcting typos in a GitHub corpus, and fuzzy-joining tables), they report model-size reductions up to

76% and throughput gains between 2.52× and 3.31× against an uncompressed Llama-3.1-Instruct-8B baseline. The prototype is on pandas, not a production OLAP engine, which matters.

Xiu et al. [3] step away from the retrieve-versus-execute axis entirely. In ByteHTAP, a system with a row-oriented TP engine and a column-oriented AP engine shares one interface. They build a RAG-backed explainer that tells a user *why* one engine was faster on a given query. Historical plan pairs get embedded by a lightweight tree-CNN classifier and stored with expert-curated explanations. At query time, the two most similar vectors are retrieved and grounded into the LLM’s prompt. The LLM never generates SQL. It never touches data. It only talks about what the engines did. Expert reviewers rated 91% of generated explanations accurate when two or more similar historical vectors were retrieved; a small user study found participants given the LLM explanation reached correct understanding in about 3.5 minutes versus 8.2 minutes from raw plan details.

The three share almost nothing structurally. The LLM lives in a different place in each. The unit of work is different cuboids, rows, plan pairs. What unites them is the underlying constraint: a vanilla LLM on warehouse-scale structured data is too slow, too expensive, and too often wrong, and each paper escapes that constraint by moving the LLM somewhere its weaknesses matter less. Section 2 places the three papers inside the wider research space. Section 3 goes into how each system actually works. Section 5 picks at the weaknesses. Section 6 is the proposal.

2 Related Work

The three papers sit in different corners of a common space. On one end are systems that use LLMs to *generate* SQL or code text-to-SQL, wrangling assistants. This line of work has well-documented problems with cross-domain generalisation [4] (cited within [1]), which is part of why Ouafiq and Saadane decide *not* to rely on generated SQL for the aggregation work and instead precompute the answers. At the other end are systems that use LLMs as *operators* inside the query pipeline, transforming data row-by-row; that is the tradition Mohammadi and Bindschaedler inherit and try to make cheap enough to be practical. Between these extremes are systems where the LLM is a commentator rather than a participant, diagnosing problems, explaining plans, helping users understand what the database did, which is the space Xiu et al. work in, and where they compare themselves directly against DBG-PT [5] (cited within [3]).

3 How Each System Actually Works

3.1 RA-OLAP: Pre-materialise, then Retrieve

RA-OLAP [1] has two phases worth separating. Cube construction happens in Apache Spark: a ReduceByKey produces an aggregate along all N dimensions, then FlatMap decomposes that parent cuboid into $N-1$ -dimensional children by projecting away one

Table 1: IOLM-DB vs. Llama-3.1-Instruct-8B baseline. Accuracy normalised so baseline = 1.0. From [2].

Workload	Baseline	IOLM-Perf	IOLM-Acc
Summarisation: size (GB)	14.98	8.48	8.48
Summarisation: rows/s	4.67	15.50	11.97
Summarisation: accuracy	1.00	0.91	1.00
Correction: rows/s	2.73	7.60	7.60
Fuzzy join: rows/s	14.92	37.72	37.72

dimension at a time, recursing down to the 0-dimensional apex. Spark’s lazy evaluation keeps intermediate DataFrames in memory until an action is triggered, so the hierarchical derivation avoids redundant disk I/O.

The second phase is where the LLM integration actually lives. Each cuboid gets identified by a bit vector indicating which dimensions participate in its GROUP BY. For a 2-dimensional cube with dimensions DY and DX, the cuboid containing both has bit vector (1, 1) and the one that rolls up DX has (1, 0). Dimension values are mapped through an encoding dictionary to small integers, and the Cassandra row key is the concatenation of the bit vector and the encoded values something like 00000011+01 for the cell {DY=Y2, DX=X2}. At query time, the user’s SELECT/GROUP BY/WHERE clauses get reverse-engineered into the corresponding cuboid ID and a scan range over row keys, so Cassandra does a partition-range scan rather than a full table scan. What does the LLM actually do here? It translates natural language into a structured query and generates context-aware explanations. It does not compute any aggregates.

3.2 IOLM-DB: Shrink the Model, Then Run It In-Line

IOLM-DB [2] assumes the LLM gets invoked from inside the SQL query, like `SELECT prompt('summarise: ' || review) FROM reviews`. At that granularity, a general-purpose model is a non-starter. The response is to compile, per query, a specialised model that still performs the task but at a fraction of the cost. Three compression techniques combine: 8-bit quantisation, structured or unstructured sparsification, and structural pruning that removes attention heads or whole layers whose contribution to the specific query is small. Calibration uses a small unlabelled sample from the query’s own input domain to tune quantisation parameters and pruning thresholds.

Two things in Table 1 the paper does not emphasise. On summarisation, the highest-throughput configuration drops normalised accuracy to 0.91, while correction and fuzzy join stay at 1.0 compression tolerance appears to be workload-dependent in a way the authors do not characterise. The other thing: generating an instance-optimised model takes, by the authors’ own account, “in the order of single-digit minutes.” Fine for a long-running analytical query; not fine for an interactive dashboard.

3.3 HTAP Query Performance Explanation

One idea in Xiu et al. [3] is more interesting. When a query hits ByteHTAP, both engines produce an execution plan, and both plans are fed to a *smart router*: a tree-CNN classifier whose original job was to predict which engine runs faster. The router is tiny: under

1 MB, inference around 1 ms. The authors repurpose it as a *plan embedder*. Because it was trained to distinguish AP from TP plans by their likely performance difference, its internal representations encode something like “performance-relevant features of a plan pair” rather than generic structural similarity. The knowledge base retrieves by performance-distinction similarity, not plan-shape similarity. That is the whole reason the system works.

The knowledge base stores (plan_pair_encoding, plan_details, execution_result, expert_explanation), where the encoding is a 16-dimensional vector. For a new query, the top-2 most similar vectors are retrieved and handed to an LLM with structured prompts that explicitly forbid comparing cost estimates from the two optimisers directly, because those costs are not on a common scale. Twenty curated queries in the knowledge base, 200 synthetic queries in the test set; expert-rated accuracy reaches 91% with two retrieved vectors and drops to 85% with only one. The authors compare against DBG-PT [5], which is effectively their own system without RAG grounding, and show that removing the grounding causes the LLM to over-emphasise columnar storage as the cause of AP’s speed and under-weight the real cause (missing indexes, forcing nested loops).

4 What Each Paper Does Well

RA-OLAP’s real strength is the encoding scheme, not the LLM framing. The bit-vector row key is a clean solution to a problem Cassandra is genuinely bad at, multi-dimensional roll-ups on a store designed for high-ingest partition scans. By collapsing the cuboid identifier and the dimension values into a single concatenated key, the authors turn an aggregation query into a prefix scan, which Cassandra does well. The reported CPU and memory reductions (44% and 62.5%) come from this, not from anything the LLM is doing, and even readers who are sceptical of the broader RAG framing can take the encoding design away as a useful contribution in its own right.

The real strength of IOLM-DB lies in its commitment to measuring what actually matters: genuine throughput on real-world workloads. Seeing summarization jump from 4.67 to 15.50 rows per second, a solid 3.31× improvement gives us something concrete. The paper also does something rarer than it should be: it names the workload-dependent trade-off explicitly, distinguishing IOLM-DB-Perf (fastest) from IOLM-DB-Acc (lossless), rather than picking a single configuration and claiming it dominates. That honesty is what makes the pandas-prototype limitation tolerable. We can see what is being traded, and that kind of clarity is useful even if the setup is partial.

The strength of Xiu et al. is the observation that a model trained for routing can be repurposed as an embedder for retrieval. This is a small idea but it solves a real problem: general-purpose embedding models would give you plan-shape similarity, which is the wrong similarity for this task. By reusing the tree-CNN router, which was already trained to discriminate on performance grounds, the authors get an embedding function tuned to the right notion of similarity essentially for free. The fact that the router is also tiny (under 1 MB, 1 ms inference) means this does not become the bottleneck. The participant study is the other thing I would highlight: reducing time-to-understanding from 8.2 minutes to 3.5 minutes is the kind of concrete user-facing result.

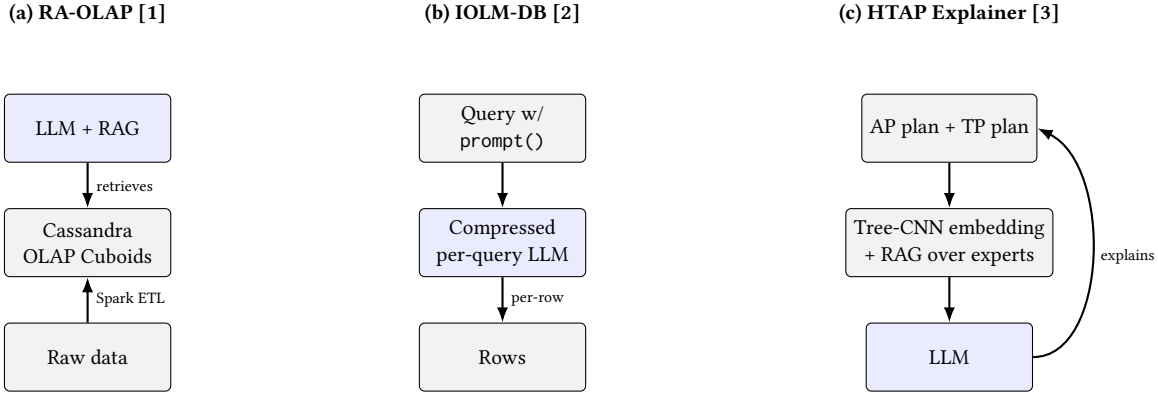


Figure 1: Three placements of the LLM relative to the analytical engine. In (a) the LLM sits *outside* the engine and retrieves pre-aggregated cuboids. In (b) it sits *inside* the query pipeline as a per-row operator. In (c) it sits *above* the engine, describing what the engine did.

5 Weaknesses and Open Challenges

Each paper has a weakness the authors are upfront about. The more interesting ones are the weaknesses the papers underplay or do not discuss.

The thing I noticed most in RA-OLAP [1] is how little space the paper gives to the LLM component, despite “LLM” being in almost every paragraph. The contribution is genuinely on the retrieval side the cuboid encoding, the Cassandra integration, the Spark pipeline, all of which is worked out in detail. The runtime does depend on an LLM that translates natural language into an OLAP query, but the paper says almost nothing about how that translation is implemented. Their Figure 7 shows LLM query-translation accuracy degrading as query complexity increases, but “complexity” is never defined on that axis and there is no benchmark comparison. So the 44% CPU and 62.5% memory savings are real, for the cube retrieval path. If the upstream LLM routes a user question to the wrong cuboid ID, those savings buy a wrong answer very efficiently.

IOLM-DB’s problems are harder to summarise cleanly. The authors are clear about the prototype being in pandas. What they do not engage with is that pandas has no cost model, no query optimiser, no parallel execution strategy, and no spill-to-disk handling, which is to say pandas has essentially none of what makes an OLAP engine an OLAP engine. Claiming the technique transfers to a production engine is plausible but the pandas evaluation does not support it; and the reported throughput numbers (baseline 4.67 rows/s on summarisation, for instance) suggest workloads in the thousands of rows rather than the millions-to-billions the abstract motivates. Separate from the prototype issue is an evaluation issue that I think matters more: the summarisation accuracy drop from 1.00 to 0.91 under the performance-optimised configuration is reported in a single number with no analysis of what kinds of inputs produced the divergences. Accuracy on a subjective task is the authors’ own judgement anyway, normalised against the uncompressed baseline which is assumed to be 1.0 by definition, a fragile setup, and one that does not tell us whether accuracy degrades linearly or falls off a cliff as compression gets more aggressive. The two issues compound: we cannot assess how the technique scales to real workloads and we also cannot assess how it fails when it does.

Xiu et al. evaluate inside a narrow box. Twenty curated queries, all from TPC-H, mainly joins and Top-N; 91% accuracy on 200 synthetic queries from the same distribution. Retrieval quality on out-of-distribution plans is unknown, and the 3.5% None rate the authors report is on in-distribution data.

One thing applies to all three. None has a feedback loop. RA-OLAP’s translation errors are not fed back anywhere. IOLM-DB’s accuracy drops are not fed back into the compression schedule. Xiu et al. come closest, experts correct wrong explanations and the corrections go into the knowledge base, but the loop is human-in-the-loop, not system-to-system. That gap is where I want to try to put something.

6 A Feedback Loop Between Explanation and Instance Optimisation

Two of these papers are closer than they might look. IOLM-DB generates a compressed LLM per query and has no systematic way to notice when that model is losing accuracy in production. The HTAP explainer produces structured, grounded natural-language diagnostics of why something behaved the way it did. The connection I want to propose is that the explainer’s output format is a supervision signal that instance optimisation is currently throwing away.

Three components make this concrete. First, an **online accuracy sentinel**: for a small random subset of the rows a compressed IOLM-DB model processes, also run the uncompressed baseline and log disagreements. Proportionally this is cheap, sampling 0.1% of rows roughly doubles cost on that slice only, leaving the 2.5–3.3× throughput improvement [2] mostly intact. Output: a stream of (input, compressed output, baseline output, disagreement) tuples.

Second, an **explanation layer that classifies disagreements**. When a disagreement shows up, an explainer in the style of [3] generates a diagnosis of why the compressed model diverged “the input contained domain terminology the pruned model lost,” or “the compressed model hallucinated a product category not in the vocabulary.” The RAG knowledge base stores disagreement cases annotated by humans the first time each class is seen, and subsequent similar disagreements get retrieved and classified automatically.

Third, a **compression feedback signal**. Classified disagreements feed back into the calibration step of the next IOLM-DB model for the same or similar query. Calibration currently samples uniformly from the query’s input distribution; under the proposal, the sample is re-weighted toward subspaces where prior disagreements concentrated. If the sentinel keeps flagging one category of input as a weak spot, the next compressed model is calibrated with more examples from that category, or pruned less aggressively on the attention heads the explainer implicated.

This could work because it does not actually change either component’s core mechanism. IOLM-DB’s compression is already calibration-driven; the proposal only changes the sample. The explainer infrastructure of [3] already transforms expert knowledge into retrieval-grounded diagnostics; here the “experts” become a sampled baseline rather than human reviewers. What is new is the loop between them.

The main thing I am unsure about is whether the signal is clean enough to actually be useful. For a subjective task like summarisation the baseline is not ground truth, it is just a stronger opinion, so the loop ends up optimising the compressed model toward imitating the baseline rather than toward correctness. The explanation step has its own error rate (85–91% on in-distribution data in [3], and likely worse on the messier task of classifying disagreement causes), and a noisy classifier is arguably worse than no classifier at all if it consistently pushes calibration in the wrong direction.

Worked example of the proposed loop. Suppose the query is the Amazon-reviews summarisation workload from [2], and IOLM-DB has produced a compressed 8.48 GB model by pruning attention heads that the calibration sample found least important. The sentinel, sampling 0.1% of rows, runs both the compressed model and the uncompressed 14.98 GB baseline on the same 2,000 reviews out of 2 million. On 37 of those the outputs disagree; on 29 of the 37, the compressed summary drops a product-category term (for example, the review mentions “DSLR lens” but the compressed summary says “camera accessory”). The explainer, prompted with the disagreement examples and retrieving similar past cases from the knowledge base, classifies these 29 as “domain-specific noun generalisation in compressed output.” That classification is the signal. For the *next* instance-optimisation pass on a summarisation query, the calibration sample is re-weighted toward reviews containing domain-specific product nouns, and pruning thresholds on the attention heads the explainer flagged are relaxed. The concrete effect would be a slightly larger compressed model, say, 9.2 GB instead of 8.48 GB, that keeps its accuracy on the category of input the sentinel identified as weak. Whether this actually recovers accuracy without eating too much of the throughput gain is the empirical question the proposal is making testable. I am not claiming this will work. I am claiming it is the smallest concrete loop you can build from the two systems as they exist today.

7 Conclusion

The three papers agree that general-purpose LLMs applied naively to warehouse-scale structured data are too slow, too expensive, and too unreliable. They disagree on what to do about it: make retrieval analytically aware and keep the LLM out of the data path [1], compress the LLM enough to put it directly on the data path [2], or keep the LLM away from the data path entirely and use it to narrate what the data path did [3]. Running underneath those disagreements is a

common gap: none of the three systems has a mechanism for detecting or correcting when it is wrong. Section 6 sketches one way to close that gap between two of them. Whether the proposal actually helps is an empirical question this report cannot answer, but the three placements are probably not really alternatives a production analytical system that genuinely cares about LLM integration will most likely want all three, applied to different parts of its workload, and how they coordinate is the open research question.

Declaration of AI use. I used Claude sonnet 4.6 for language polishing, grammar correction, and improving clarity of expression. All intellectual contributions, including the analysis, synthesis, arguments, and conclusions, are solely my own.

References

- [1] E. M. Ouafiq and R. Saadane. Retrieval-Augmented OLAP: Generative AI Architecture for Smart Systems & Equipment. In *AAAI Summer Symposium Series (SuSS-25)*, 2025.
- [2] B. Mohammadi and L. Bindschaedler. The Case for Instance-Optimized LLMs in OLAP Databases. In *DOLAP 2025: 27th International Workshop on Design, Optimization, Languages and Analytical Processing of Big Data*, 2025. arXiv:2507.04967.
- [3] H. Xiu, L. Zhang, T. Zhang, J. Yang, and J. Chen. Query Performance Explanation through Large Language Model for HTAP Systems. In *EDBT 2026*, pages 103–109. arXiv:2412.01709, 2024.
- [4] X. Chen. Exploring underexplored limitations of cross-domain text-to-SQL generalization. arXiv:2109.05157, 2021. Cited within [1]; not read firsthand.
- [5] V. Giannakouris and I. Trummer. DBG-PT: A large language model assisted query performance regression debugger. *Proc. VLDB Endow.* 17(12), 2024. Cited within [3]; not read firsthand.